PromptMark: A Prompt-Guided Iterative-Feedback Framework for Source Code Watermarking

Keywords: Source Code Watermarking, Code Provenance, Prompt-based Watermarking, Black-Box watermarking

Abstract: Watermarking has become a crucial technique for ensuring provenance and accountability in AI-generated source code. As large language models (LLMs) are increasingly integrated into development workflows, reliable attribution remains challenging. In practice, most developers rely on commercial LLM APIs operating under black-box constraints, making existing approaches that require access to the decoding process less feasible for real-world integration. To address this limitation, we propose PromptMark, a black-box, prompt-guided watermarking framework that embeds invisible yet statistically detectable signals into generated code via structured input instructions. The method steers models toward subtle identifier and comment naming patterns while preserving the functional correctness and structural integrity of the generated code. Detection is performed using statistical tests designed to remain reliable across varying code lengths and model outputs. The embedding is further refined through an iterative feedback loop, where prompts are updated based on watermark detection scores. Experiments on the MBPP and HumanEval benchmarks show that PromptMark consistently achieves strong watermark detectability while maintaining high code correctness, outperforming baseline approaches.

1 INTRODUCTION

Watermarking refers to the process of embedding a hidden but verifiable signal into digital content, enabling author attribution or provenance tracking. It has long been recognized as an effective technique for protecting the ownership and integrity of digital artifacts (Collberg and Thomborson, 2002). While traditional watermarking methods have been widely applied to images, audio, and text (Hayes and Danezis, 2017; Liu et al., 2024), extending this concept to source code introduces distinct challenges. Source code is highly structured and syntactically rigid, where even minor modifications can break compilation or alter semantics. At the same time, Large Language Models (LLMs) are being widely used by the develoeprs to generate and modify code, making reliable attribution of AI-generated programs an important concern. Despite this concern, designing effective watermarking approach while preserving code correctness remains a challenging task.

In practical development settings, most users interact with LLMs through commercial API-based services that operate under strict black-box constraints, where only input and output are accessible (Bistarelli et al., 2025). This limitation restricts direct control over the generation process, making it difficult to embed reliable watermark signals without compromising code correctness. Existing watermarking approaches are largely designed under white-box assumptions, where access to model internals enables manipulation of token probabilities or decoding strategies (Lee et al., 2024; Guan et al., 2024). While effective in controlled environments, their applicability in practical deployment settings remains limited.

Under these constraints, watermarking methods should be designed that do not require access to model internals. Prompt-based methods provide a practical alternative, as they utilize input instructions to influence generation behavior without modifying the underlying model (Dasgupta et al., 2024). In particular, they exploit the instruction-following capabilities of modern LLMs to steer outputs toward desired patterns. However, effectively embedding watermark signals through prompts remains challenging, as it requires balancing the inclusion of detectable patterns with the preservation of syntactic correctness and functional behavior of the generated code.

Several studies have explored different watermarking techniques for AI-generated code. Notable approaches influence token selection through entropy-thresholding (Lee et al., 2024), enforce grammar-guided constraints to support multi-bit embedding (Guan et al., 2024), or bias token prediction by manipulating non-syntax tokens (Kim et al., 2025). Recent studies also explore attention-based triggers to determine watermarking location for improved robustness (Li et al., 2025). Although effec-

tive, their applicability in black-box settings is limited due to reliance on model internals.

To address these limitations, we propose **PromptMark**, a prompt-guided black-box watermarking framework for source code. This framework operates in three phases. In the first phase, PromptMark constructs a system prompt that guides the model to generate identifiers and comments whose initial characters follow a predefined adaptive green-list, which serves as the watermark signal. This list is derived through frequency-based stratification of identifier initials observed in human-written code. In the second phase, the model generates code following an iterative feedback loop that checks for both functional correctness and sufficient watermark strength. When either condition is not satisfied, a summarized feedback log is incorporated into the subsequent prompt to refine the output. Finally, the detection phase identifies watermarks by extracting the initial characters from identifiers and comments in the generated code. A p-value is then computed based on the observed frequency of green-list characters to quantify the likelihood of watermark presence.

Through experiments on the MBPP and HumanEval datasets, we evaluated PromptMark on Python code using two closed-source models: Gemini 2.5 Flash and Claude Sonnet 4. We focus on two core aspects: effectiveness and code quality of the generated watermarked code. Across different configurations, the iterative-feedback method with an adaptive green-list achieves strong detection performance, with AUROC ranging from approximately 0.90 to 0.98. At low false positive rate (1%–5% FPR), the method showed high true positive rates, exceeding 90% in the majority of configurations and reaching above 95% in favorable settings. In terms of code quality, the framework preserves functional correctness around 90% of the cases, while maintaining near similar CodeBLEU (Ren et al., 2020) scores to non-watermarked code.

2 PROBLEM DEFINITION AND RELATED WORK

This section formalizes the problem of watermarking and reviews existing studies for establishing authorship and origin of software artifacts. We first introduce the concept of code provenance, followed by two primary paradigms: active source code watermarking and passive authorship attribution. We then discuss existing techniques within these paradigms and highlight their limitations in practical deployment settings.

2.1 The Concept of Code Provenance

Code provenance is defined as the verifiable, auditable record of the origin, authorship, and modification history of software. It functions as a "chain of custody" for code, ensuring that every artifact in production can be traced to specific, authorized commits and builders. In environments where AI and humans now work as co-authors, the origin of code cannot be inherently trusted. Rather, the identity of the coder or maintainer becomes the anchor of trust. Accordingly, every commit, whether written by a human or generated by an AI, must be signed and bound to a verified identity to ensure accountability rather than restricting automation. This principle is formalized in frameworks such as Supply-chain Levels for Software Artifacts (SLSA), which, under policies influenced by Executive Order 14028, requires verifiable proof of authorship and build integrity to sell software to the U.S. government (Gu, 2025).

Previous studies (Xiang et al., 2025) address provenance from two different angles: watermarking (Guan et al., 2024) and authorship attribution (Gurioli et al., 2025). Below we explain each of them in detail:

2.2 Watermarking

Watermarking is a way to hide a secret signal inside a carrier, such as text or images. It is used to prove ownership or check whether the content is authentic (Collberg and Thomborson, 2002). When we apply this to a generative model, it involves changing or biasing the model's token prediction process to leave a detectable mark in the generated output.

Source code watermarking, on the other hand, is more challenging than natural language because we must additionally ensure that the watermarked version of the code is functional and executable. (Guan et al., 2024). Unlike a story or an essay, code is fragile: a single wrong character can break functionality or even alter the program logic for which the code was initially written. (Lee et al., 2024). Therefore, a code watermark must modify the code in such a way that the watermarking signal is hidden and the logic and functionality remain intact.

Formally, a watermarking system for a model $\mathcal{M}$ consists of two parts $(\mathcal{G}, \mathcal{D})$ (Yang et al., 2024):

1. **Generation ($\mathcal{G}$):** The system takes a prompt x and a secret key k to produce watermarked code y_w following the Equation 1:

$$\mathcal{G}(x, \mathcal{M}, k) \rightarrow y_w \tag{1}$$

   During this step, the model is biased to favor certain tokens determined by the key k (Kirchenbauer et al., 2023; Lee et al., 2024).

2. **Detection ($\mathcal{D}$):** The system takes a suspect code snippet $\hat{y}$ and the secret key k to decide whether the code is watermarked. The detector computes a continuous statistical score representing the strength of the embedded signal, defining the detection as a formal hypothesis testing problem:
   - H_0: The code $\hat{y}$ is generated without the knowledge of k (not watermarked).
   - H_1: The code $\hat{y}$ is generated with the knowledge of k (watermarked).

   The detection function provides a binary decision by comparing this computed score against a predefined threshold η:

$$\mathcal{D}(\hat{y}, k) \rightarrow d, \quad d \in \{\text{true}, \text{false}\}. \tag{2}$$

The detector returns *true* (rejecting H_0) when the score is $> \eta$, indicating that the code is "watermarked." It returns *false* when the score is $<= \eta$, suggesting the observed pattern is likely due to random chance. As the detection process strictly verifies the binary presence or absence of a signal, it is formally categorized as *zero-bit* watermarking (Zhao et al., 2025). While *multi-bit* schemes extend this concept by embedding recoverable payloads within the content. Our research here focuses exclusively on the **zero-bit** approach.

2.3 Authorship Attribution

Authorship attribution is a forensic method used to figure out who wrote a document based on how it was written. This process relies on "Code Stylometry" where the unique style of writing a code provides the trace of the author. These stylistic traces are the fingerprints of the programmers that separate one's work from others (Gurioli et al., 2025). Researchers look for specific style features in the code to find these fingerprints. This includes lexical features like how variables are named or how comments are written (Park et al., 2025). It also includes structural features. These can be patterns like how deep loops are nested or the specific way the code logic is organized (Paek and Mohan, 2025). Some modern systems even use neural networks to learn these patterns automatically (Yin et al., 2025; Xu et al., 2024).

2.4 Related Work

A series of studies has investigated in-process watermarking for code generation, where watermark signals are embedded during the decoding process. Early efforts include SWEET (Lee et al., 2024), which applies entropy-based logit adjustments to bias token selection toward statistically identifiable patterns. CODEIP (Guan et al., 2024) extends this idea by leveraging programming-language grammar to guide token prediction and support multi-bit watermark embedding. To improve naturalness and functional integrity, STONE (Kim et al., 2025) limits watermarking to non-syntax tokens such as identifiers and literals. Beyond lexical manipulation, CodeGuard (Li et al., 2025) introduces an attention-based approach that activates watermarking through learned triggers and harmonic character patterns. MCGMark (Ning et al., 2024) proposes an encodable watermarking approach for tracing malicious code. They manipulate vocabulary selection and skip specific tokens to enhance robustness. RoSeMery (Zhang et al., 2025) adopts an ML/crypto co-design, embedding multi-bit signals through encoder-guided renaming and structural modifications while safeguarding watermark keys. Although diverse in technique, all of these methods rely on access to model internals, which makes them limited to white-box approaches.

Separately, a growing body of research examines prompt-level watermarking, where watermark signals are embedded solely through natural-language instructions. In-context watermarking (ICW) (Liu et al., 2025) demonstrates that lexical preferences can be imposed through system prompts, while prompt-guided multi-LM frameworks (Dasgupta et al., 2024) show how watermark instructions, generation, and detection strategies can all be produced through coordinated prompting. However, these studies focus primarily on natural-language generation rather than source code. CodeMark-LLM (Xu et al., 2025) and ACW (Li et al., 2024) adopts a post-hoc strategy that uses LLMs to refactor existing code via pre-defined semantic-preserving transformations to embed watermarks. However, CodeMark-LLM is a multi-bit watermarking approach and requires access to the original code for verification.

Despite these advancements, prompt-based watermarking for code-generation tasks in black-box, API-only settings remains largely unexplored. To the best of our knowledge, no prior work has investigated whether carefully crafted prompts can reliably embed detectable watermark signals in source code.

3 METHODOLOGY

We propose a *prompt-guided watermarking framework* for source code that embeds imperceptible yet statistically detectable watermarks into code generated by LLMs. Prior studies (Lee et al., 2024; Kim et al., 2025) assume white-box access and embed signals by directly biasing token logits during decod-

Table 1: Identifier types and descriptions

Identifier Type	Description
Function Names	User-defined function names
Class Names	User-defined class names
Variable Names	Local/global variable identifiers
Function Arguments	Parameter names in function definitions
Attribute Names	Object attributes or method references
Loop Variables	Iterator variables in loop constructs

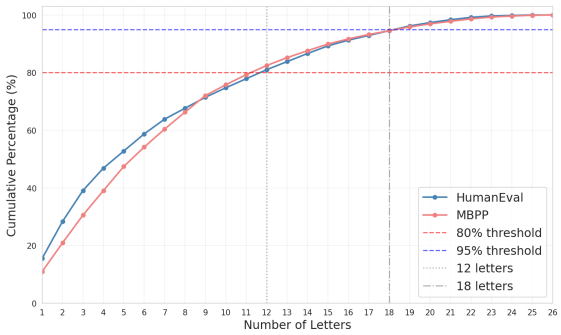

Figure 1: Combined cumulative initial letter distributions (HumanEval / MBPP)

ing. In contrast, we target closed-source models like Gemini, Claude etc. where logit manipulation and decoder modification are unavailable. Moreover, large-scale parameter updates or fine-tuning are often restricted or economically impractical. Consequently, under black-box constraints, watermarking must rely on input-level control such as prompt instructions to steer the generation process which motivates our approach. Our three-step method embeds watermarks without accessing model internals rather leverage the instruction following capabilities of LLMs.

3.1 Prompt Construction

In this step, we construct a system prompt that biases the model toward producing watermark-carrying tokens specifically, identifiers and comments whose *initial characters* belong to a predefined *green-list*. Building on the general "green-list/red-list" watermarking paradigm introduced in WLLM (Kirchenbauer et al., 2023), we define the *green-list* as the subset of allowed initial characters that encodes the watermark signal. The complementary *red-set* contains characters that the model is instructed to avoid as initial characters for the targeted tokens. Enforcing this constraint enables a statistically detectable skew in initial-character frequencies that distinguishes watermarked outputs from non-watermarked ones.

However, identifier names in source code cannot be modified arbitrarily as it might break dependencies, public interfaces, or library conventions. Therefore, we restrict watermarking to a set of "safe" Python identifier categories (see Table 1) that can be altered without affecting functionality. In order to generate the green and red sets, we perform frequency-based stratification over the initial characters of identifiers observed in human-written code (Lee et al., 2024). Finally, to increase embedding capacity beyond identifiers, we additionally watermark comments using the same initial-character constraint. The complete prompt is presented in Figure 3.

3.2 Frequency-Based Token Stratification

To embed watermark without introducing unnatural or forced identifier name, we analyze the distribution of identifier-initial letters in human-written Python code using the MBPP (Austin et al., 2021) and HumanEval (Chen et al., 2021) datasets. This allows the watermark to bias initial characters while preserving semantically meaningful and human-like naming conventions. The observed rank-frequency distribution (f) of the character initials follows a Zipf-like pattern (Piantadosi, 2014), where a relatively small subset of letters (12-18) accounts for a large proportion of identifier initials (see Figure 1).

Since both MBPP and HumanEval primarily contain short-to-medium length programs with a limited number of unique identifiers, the expected natural occurrence rate γ must remain moderate to ensure statistical separability between watermarked and non-watermarked code. If γ is too high, most identifiers would naturally satisfy the green-list constraint, reducing discriminative power in the hypothesis test. To determine an appropriate candidate-pool size, we compute the empirical baseline probability γ for varying values of K (number of top frequent letters).

To determine the suitable range, we conducted experiments over a broader spectrum of K values. The results consistently exhibit a monotonic increasing trend in γ with respect to K, confirming that larger candidate pools lead to higher baseline probabilities and reduced statistical separability. Thus, given the moderate code length in MBPP and HumanEval datasets, we select a candidate-pool range that keeps γ within a balanced interval, preserving both naturalness and statistical detectability. Accordingly, we define the candidate set C as the top-K most frequent initial characters:

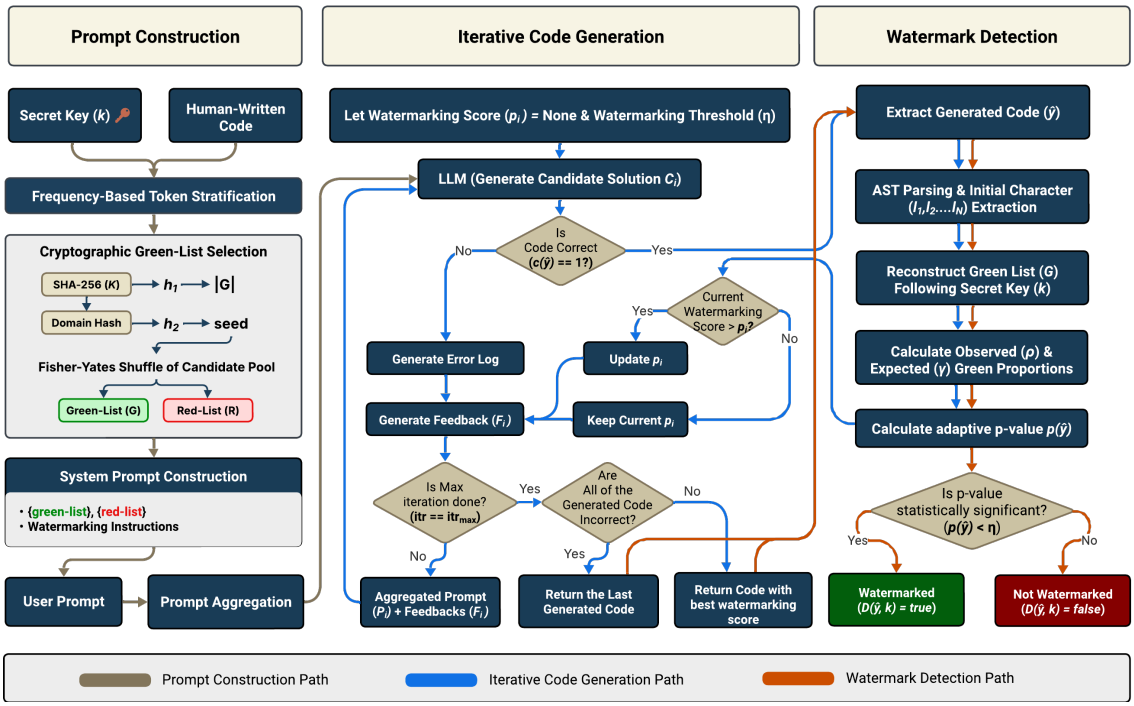

Figure 2: Overview of the PromptMark framework.

You are an expert Python programmer and code reviewer. Your task is to write a function that adheres to specific identifier naming rules and generates code based on a given problem statement.

Additional Instruction:

Green Letter List: {green_words}

Red Letter List: {red_words}

Command:

Generate code following the given instructions:

1. Green Letter Enriched Identifier:

When generating identifiers (e.g., local variables, parameters, helper functions, attributes, temporary variables) & comments, start words with letters from the **Green Letter List**. Use examples as naming references.

2. Correct & Relevant:

Generate correct code following the problem statement.

3. About comments:

Add brief comments only for complex logic. Avoid tokens starting with **Red List** letters.

4. About Method Name:

Use the exact method name from the given test case.

5. Warning:

Do not mention or explain renaming rules in output.

6. Others:

Provide explanation in exactly three bullet points after the code. If incorrect, add corrected code after explanation. Never include test cases, explanations, or assertions inside code blocks.

Figure 3: System prompt used to constrain and bias the code generation process.

$$C = \text{top}_K(f), \quad K \in [10, 12]. \tag{3}$$

This range maintains a moderate baseline green-letter probability ($\gamma \approx 0.52$–0.62), allowing reliable segregation between watermarked and non-watermarked codes while maintaining consistency with natural identifier distributions. While other values of K follow similar trends, this interval provides the best trade-off between embedding strength and false positive control in our setting. However, in practical deployments, the baseline probability γ should ideally be estimated from the target developer's codebase to reflect their natural identifier usage patterns.

3.2.1 Cryptographic Derivation of the Green-List

To avoid fixed cardinality and predictable identifiers, we derive the green-list size $|G|$ and its specific letter composition directly from a 128-bit secret key k. The complete deterministic extraction procedure is summarized in Algorithm 1.

Uniform Sampling of Candidate-Pool Size. We utilize the SHA-256 hash function to approximate uniform randomness (Line 1). By extracting the first 32 bits of the hashed key, we obtain a uniformly distributed continuous variable in $[0, 1)$ (Line 2). This variable is linearly scaled to uniformly sample the cardinality $|G|$ from the admissible bounds $[|G|_{\min}, |G|_{\max}]$ (Line 3). This ensures that every permissible size for the green-list has an equal probability of being selected, preventing predictable patterns

in the watermark's structure.

Domain-Separated Unbiased Permutation. To determine which specific letters form G, we compute a second, domain-separated hash (Line 4) to generate a 64-bit integer seed (Line 5). This seed drives a Fisher-Yates shuffle (Knuth, 1997) over the empirically grounded candidate pool C (Line 6). The Fisher-Yates algorithm guarantees an unbiased permutation, ensuring every possible ordering of C is produced with equal probability. The first $|G|$ letters of this permuted list form the green-list G, while the remaining letters form the complement red-set R (Lines 7–8).

Algorithm 1 Cryptographic Green-List Construction

Input: Secret key k, Candidate pool C, Cardinality bounds $|G|_{\min}, |G|_{\max}$
Output: Green-List G, Red-List R

1: $h_1 \leftarrow \text{SHA-256}(k)$
2: $u_1 \leftarrow \text{bytes_to_int}(h_1[0:4])/2^{32}$ ▷ extract uniform variable $\in [0,1)$
3: $|G| \leftarrow \lfloor |G|_{\min} + u_1 \cdot (|G|_{\max} - |G|_{\min} + 1) \rfloor$
4: $h_2 \leftarrow \text{SHA-256}(k \parallel \text{“green-set-selection”})$
5: $\text{seed} \leftarrow \text{bytes_to_int}(h_2[0:8])$ ▷ generate domain-separated seed
6: $\mathtt{p} \leftarrow \text{Fisher-Yates-Shuffle}(C, \text{seed})$ ▷ unbiased permutation of C
7: $G \leftarrow \{\mathtt{p}[0], \mathtt{p}[1], \ldots, \mathtt{p}[|G|-1]\}$
8: $R \leftarrow C \setminus G$
9: **return** G, R

The proposed green-list generation strategy ensures reproducibility, security, and naturalness. Given the same secret key k, the same green-list G is deterministically generated, enabling consistent watermark embedding and verification in black-box settings. The use of the non-invertible SHA-256 hash function makes the mapping from k to G cryptographically secure, as recovering the 128-bit seed would require approximately 2^{128} operations. Additionally, selecting green-list characters from frequently used identifier initials preserves semantically meaningful and human-like naming patterns while still shifting the distribution enough to create a detectable watermark signal.

3.3 Iterative Code Generation

The watermark embedding process is designed as a feedback-driven iterative code generation process that jointly focuses on two primary objectives: **functional correctness** and **watermark fidelity** (see Figure 2). Here, watermark fidelity denotes how strongly and consistently the generated code adheres to the intended watermarking pattern without compromising correctness. When the constructed prompt is passed to the LLM, it generates candidate code following the watermarking instructions provided via the system prompt. Formally, at iteration $i \in [1, Itr_{\max}]$, the model generates a candidate solution C_i guided by prompt P_i. The evaluation of each candidate is defined as

$$\text{Eval}(C_i) = (c_i, p_i), \tag{4}$$

where c_i is the correctness score based on unit-test execution ($c_i = 1$ if all tests pass, otherwise 0), and p_i is the watermark fidelity score which is a p-value score as calculated in the detection phase, Section 3.4.

After each generation, the LLM produces feedback F_i to improve any unmet evaluation criteria. If C_i fails the functional tests, the summarized error is returned as feedback. If the code is correct but the watermark score is below the threshold, the feedback instructs the model to use more identifiers starting with green-list characters. The next prompt is formed as $P_{i+1} = f(P_i, F_i)$, where $f(\cdot)$ denotes the prompt-update operation that appends the feedback into the next prompt. This loop continues until both objectives are met or the iteration limit $Itr_{\max}$ is reached, forming the stopping criterion of iteration.

If no candidate satisfies both constraints, we return the candidate with best watermark fidelity score p_i, representing the closest achievable watermark signal. This preserves the strongest available watermark signal instead of returning an arbitrary result. Thus, the iterative-feedback method steers the model toward candidate solutions that jointly improve watermark strength while preserving the correctness of the generated code.

3.4 Watermark Detection

The watermark detection is treated as a statistical test that checks whether a generated code shows a noticeable shift in the distribution of identifier initials compared to human-written code. This process is model-agnostic and works in a black-box setting without access to model parameters. Let $\hat{y}$ denote the final code produced by the embedding process. The code is parsed into an Abstract Syntax Tree (AST) to extract all identifiers. From each identifier t_i, we obtain its first alphabetic letter l_i, normalized to lowercase. Let N be the total number of extracted initials, forming the sequence $L = [l_1, l_2, \ldots, l_N]$.

Let I denote the number of initials belonging to the green-list G. The observed proportion is $\rho = I/N$, while the expected proportion γ is estimated from human-written code. Watermark detection is framed

as a one-sided hypothesis test as:

$$H_0 : \rho <= \gamma \quad , \quad H_1 : \rho > \gamma,$$

Here H_1 indicates a statistically higher occurrence of green-initial identifiers. To ensure reliable detection across varying code lengths, we compute an adaptive **p-value** defined as

$$p = \begin{cases} \sum_{i=I}^{N} \binom{N}{i} \gamma^i (1-\gamma)^{N-i}, & \text{if } N < 30, \\ 1 - \Phi\left(\frac{I - N\gamma}{\sqrt{N\gamma(1-\gamma)}} \right), & \text{otherwise,} \end{cases} \tag{5}$$

where $\Phi(\cdot)$ denotes the cumulative distribution function of the standard normal distribution. The p-value estimates the probability of observing at least I green-initial identifiers out of N under the null rate γ. We refer to it as an "adaptive" p-value because it switches between the exact binomial test ($N < 30$) and the normal approximation ($N \geq 30$) to ensure accurate and stable estimation, as commonly done in statistical detection and anomaly analysis (Li, 2011).

For evaluation, we define a monotonically increasing ranking statistic $S = -\log_{10}(p)$, where larger S indicates stronger evidence of watermarks. This transformation improves dynamic range for small p-values and is used to compute the AUROC score.

To conduct the hypothesis test, we directly compare the adaptive p-value $p(\hat{y})$ against a predefined significance threshold η. Let $c(\hat{y}) \in \{0,1\}$ denote the functional correctness indicator, where $c(\hat{y}) = 1$ if the generated code passes all unit tests and 0 otherwise. Having the secret key as k, the final detection decision, illustrated in Figure 2, is defined as:

$$D(\hat{y}, k) = \begin{cases} \text{true}, & \text{if } c(\hat{y}) = 1 \wedge p(\hat{y}) < \eta, \\ \text{false}, & \text{otherwise.} \end{cases} \tag{6}$$

Thus, following Equation 6, a code is classified as watermarked only if it is functionally correct and its p-value falls below the significance threshold η.

4 RESULTS AND EVALUATION

We evaluate PromptMark across multiple watermarking strategies and two benchmark datasets, MBPP and HumanEval, using Gemini 2.5 Flash and Claude Sonnet 4. All model outputs were generated using the providers' default hyperparameters: temperature=1.0, top-p=0.95, top-k=64. The baseline methods were also evaluated under the default settings provided in their work. Followings are the research question we investigated through our experiments:

- **RQ1 (Effectiveness):** How effective is PromptMark in embedding and detecting source code watermarks?
- **RQ2 (Code Quality):** What impact does PromptMark have on code quality?

4.1 Evaluation Metrics

We evaluate PromptMark using functional correctness (Pass%), watermark detectability (AUROC, TPR, FPR, Tx%F), and code quality (CodeBLEU) metrics. These metrics are widely used in watermark-related tasks, as noted in previous studies (Lee et al., 2024; Kim et al., 2025).

Pass% (Functional Correctness). Pass% measures the percentage of generated programs that pass all unit tests:

$$\text{Pass\%} = \frac{\text{\# Correct Solution}}{\text{Total Problems}} \times 100 \tag{7}$$

Here this metric reflects whether watermark embedding preserves program functionality.

True Positive Rate (TPR). TPR measures the proportion of watermarked samples correctly identified:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{8}$$

where TP denotes watermarked code correctly detected as watermarked, and FN denotes watermarked code incorrectly classified as non-watermarked. In our scenario, a higher TPR means the detector successfully recognizes embedded watermarks.

False Positive Rate (FPR). FPR measures the proportion of non-watermarked samples incorrectly classified as watermarked:

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \tag{9}$$

where FP denotes clean (non-watermarked) code falsely detected as watermarked, and TN denotes correctly rejected clean samples. A low FPR is critical to avoid falsely attributing watermark ownership to benign code. We also measure detection performance under strict false-positive rate such as T@1%F and T@5%F where T@x%F = TPR subject to FPR $\leq x\%$

AUROC (Area Under the ROC Curve). AUROC measures the overall separability between watermarked and non-watermarked samples across all decision thresholds:

$$\text{AUROC} = \int_0^1 \text{TPR}(t)\, d(\text{FPR}(t)). \tag{10}$$

Table 2: Performance comparison of watermarking methods across MBPP and HumanEval datasets (↑ higher or ↓ lower indicates the best values for each metrics). Best results are in **bold**; grayed rows denote our final methods

Models	Method	MBPP						HumanEval					
		Pass% ↑	Avg Itr. ↓	AUROC ↑	T@1%F ↑	T@5%F ↑	CodeBLEU ↑	Pass% ↑	Avg Itr. ↓	AUROC ↑	T@1%F ↑	T@5%F ↑	CodeBLEU ↑
Qwen	SWEET	43.83	-	0.920	0.440	0.720	0.420	26.95	-	0.810	0.450	0.540	0.310
Gemini	ICW	94.44	-	0.512	0.045	0.072	0.440	86.70	-	0.400	0.010	0.040	0.480
	No Watermark	64.77	-	0.577	0.035	0.106	0.420	85.50	-	0.530	0.040	0.090	0.480
	Static	98.25	-	0.922	0.398	0.597	0.440	90.00	-	0.810	0.160	0.350	0.490
	Refactoring-Based	99.45	-	0.879	0.111	0.422	**0.910**	95.63	-	0.890	0.210	0.510	**0.500**
	Comment-based	95.01	-	0.925	0.418	0.686	0.440	88.90	-	0.850	0.330	0.510	0.470
	Iterative	**99.78**	**1.20**	0.974	0.707	0.891	0.460	**97.86**	**1.90**	0.950	0.750	0.830	0.480
	Iterative-AGL	96.79	1.90	**0.994**	**0.967**	**0.979**	0.460	88.90	1.93	**0.970**	**0.900**	**0.900**	0.480
Claude	ICW	92.40	-	0.770	0.020	0.140	0.500	94.95	-	0.740	0.010	0.120	0.490
	No Watermark	63.96	-	0.416	0.040	0.060	0.500	96.28	-	0.362	0.018	0.030	0.490
	Static	86.73	-	0.899	0.260	0.560	0.460	95.64	-	0.773	0.240	0.410	0.490
	Refactoring-Based	**99.67**	-	0.723	0.080	0.080	**0.650**	**97.86**	-	0.693	0.013	0.15	**0.790**
	Comment-based	90.00	-	0.892	0.390	0.600	0.470	91.86	-	0.855	0.420	0.590	0.490
	Iterative	97.27	**1.33**	**0.989**	0.900	0.960	0.460	97.32	**1.31**	0.903	0.563	0.680	0.500
	Iterative-AGL	93.60	2.50	0.987	**0.870**	**0.940**	0.470	94.00	1.83	**0.930**	**0.640**	**0.710**	0.480

An AUROC of 0.5 indicates random guessing, while values close to 1.0 indicate strong discriminative capability.

CodeBLEU (Code Quality). CodeBLEU evaluates structural and semantic similarity between generated and reference code:

$$\text{CodeBLEU} = \alpha \cdot \text{Ngram} + \beta \cdot \text{WeightedNgram} + \gamma \cdot \text{AST} + \delta \cdot \text{Dataflow}, \quad (11)$$

where $\alpha, \beta, \gamma, \delta$ are weighting coefficients.

Recent findings by (Mahmud et al., 2025) demonstrated that AST and Dataflow components most accurately capture structural similarity and program-level semantics of code. Therefore, we adopt their recommended weighting configuration ($\gamma = \delta = 0.5$) and proposed CodeBLEU formulation as follows:

$$\text{CodeBLEU}_{\text{reported}} = \gamma \cdot \text{AST} + \delta \cdot \text{Dataflow}. \quad (12)$$

4.2 Empirical Evaluation

Answer to RQ1. Following SWEET and WLLM, we define watermarking effectiveness as the combined ability to (i) embed a statistically detectable signal in generated code and (ii) maintain high True Positive Rates (TPR) while keeping False Positive Rates (FPR) low. Table 2 reports the full comparison across all baselines and proposed variants.

To systematically evaluate PromptMark, we structured the experiments in three stages. First, we compared our approach against two representative baselines: SWEET and In-Context Watermarking (ICW). Second, we conducted an ablation study of several prompt-driven watermarking variants in order to analyze the impact of different design choices. Finally, after identifying the most effective configuration, we evaluated the finalized framework on the corresponding datasets.[1]

[1]Replication package: https://anonymous.4open.science/r/promptmark-7AB4

Baseline Assessment. Among the baseline methods, the *Initials In-Context Watermarking (ICW)* approach performs the worst, achieving an AUROC of only 0.51. Treating source code as plain natural text fails to bias identifier distributions sufficiently to produce a detectable watermark signal. In contrast, the SWEET watermark achieves higher detectability but significantly degrades functional correctness. This degradation occurs because SWEET perturbs token logits during decoding without enforcing syntax-aware constraints. Since source code is structurally rigid, even small token-level perturbations can corrupt critical syntactic elements, and thus produce code with invalid syntax.

Evaluation of Prompt-Based Variants. We next evaluated three prompt-driven watermarking variants to understand how prompt design influences watermark embedding.

The *Static Watermarking* variant splits identifier initials into fixed green-list and red-list sets. This approach improves detectability (AUROC 0.92), indicating that identifier prefix constraints can bias the token distribution. However, the detection rate remains limited (T@1%F = 0.39), suggesting insufficient watermark coverage.

The *Refactoring-Based* variant attempts to embed watermarks by providing the solution code and asking the model to refactor it. However, this strategy performs worse than Static watermarking in terms of detectability. LLMs tend to preserve existing identifiers when provided with full source code. It rarely performs any structural edits following specified constraints which is consistent with prior findings that LLMs typically avoid modifying existing tokens unless strongly instructed (Chen et al., 2025).

The *Comment-Based* variant extends the watermarking channel by allowing comments to follow the green-list constraint. This modification partially alleviates the limited identifier issue and improves detection performance, particularly at relaxed thresholds

(T@5%F = 0.68). However, the overall watermark signal remains lower than desired.

Taken together, these observations demonstrate single-prompt watermarking strategies are insufficient for achieving both strong detectability and stable performance.

Impact of Iterative Feedback. To address the limitations of single-pass prompting, an *Iterative Watermarking* strategy is adopted, incorporating feedback during generation. Inspired by recent findings that iterative self-refinement improves instruction-following behavior of LLMs (Dong et al., 2025), our framework repeatedly evaluates candidate outputs and provides corrective feedback to the model to self-evalute its own response. This iterative mechanism substantially strengthens watermark detectability. On average, the model requires approximately 2 iterations to produce a valid watermarked program. The resulting outputs achieve significantly improved separability, with AUROC approaching 0.97 and higher detection rates under strict FPR constraints.

However, we noticed that the identifiers generated under strict green/red constraints often appear unnatural or forced, reducing code readability.

Adaptive Green-List Strategy (AGL). To address the naturalness issue, we introduced an *Adaptive Green-List* derived from the empirical identifier-initial distribution observed in human-written code (see Section 3.2). By selecting watermark tokens from high-frequency initials, the model can generate identifiers that remain semantically meaningful and stylistically natural. Integrating this adaptive strategy with iterative watermarking (Iterative-AGL) produces the strongest overall detection performance. In particular, the Gemini-MBPP setting achieves the highest separability (AUROC = 0.994) with strong detection under strict constraints (TPR = 0.967 at 1% FPR).

While the Gemini-MBPP configuration achieves the highest scores, the other model-dataset combinations show only minor variations in performance, with AUROC values consistently remaining above 0.90 across all settings. In contrast, the baseline methods exhibit a larger performance drop under these settings, as reflected in Table 2. These variations mainly arise from differences in dataset characteristics and model-specific generation behavior. MBPP primarily consists of short utility-style programming tasks, whereas HumanEval focuses on more complex algorithmic problems. As a result, HumanEval solutions typically contain fewer user-defined identifiers and comments, which slightly reduces the watermark embedding capacity compared to MBPP. Despite these structural differences between the datasets, our proposed method consistently achieves strong watermark detectability across both benchmarks, supporting RQ1.

Answer to RQ2. Across both datasets and models, PromptMark preserves functional correctness while embedding watermark signals. The correctness (Pass%) remain consistently high across all configurations scoring up to 97% on HumanEval and 99% on MBPP. This indicates that the watermarking constraints do not collide with program logic or execution behavior, and the generated programs remain functionally valid.

We measured the structural similarity among the generated code using CodeBLEU scores. Here, the Refactoring-Based variant produces the highest CodeBLEU scores overall (up to 0.91 for Gemini and 0.79 for Claude). This behavior is expected as the model receives the reference solution and tends to preserve the original structure rather than actively modifying identifiers to embed watermarks. As a result, the generated code remains highly similar to the ground-truth implementation, leading to inflated structural similarity while providing weaker watermark signals.

On the other hand, CodeBLEU remains stable across watermarking variants. Among the prompt-guided approaches, the Gemini-MBPP configuration achieves the highest CodeBLEU score (0.46), while Claude-HumanEval reaches a comparatively higher structural similarity (0.50), which closely aligns with the non-watermarked results (shown as the *No Watermark* method). These results indicate that the watermarking process preserves control-flow structure, data dependencies, and syntactic organization of the generated code across different models and datasets.

Overall, our approach consistently outperform baseline approaches while maintaining balanced functional correctness and structural integrity of the generated code, thereby supporting RQ2.

5 DISCUSSION

In this section, we analyze PromptMark's capability for model-specific identification, its resilience to code transformations, and the trade-off between iteration cost and performance.

5.1 Model-Specific Watermarking

In our framework, the watermark is generated based on a secret key k. Let the green-list be defined as:

$$G = \text{GenGreen}(k),$$

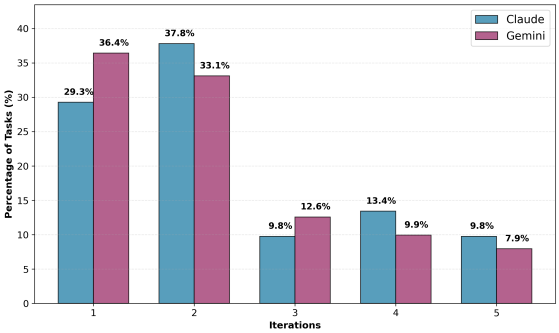

Figure 4: Iteration distribution across Claude and Gemini

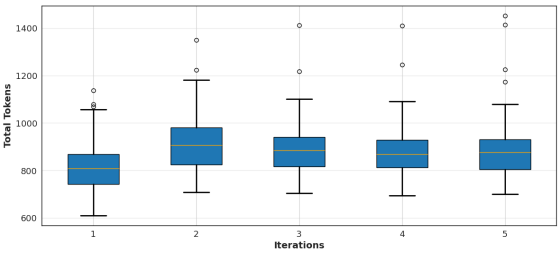

Figure 5: Box Plot of Total Tokens (Input+Output) Per Iteration

where GenGreen$(\cdot)$ denotes the deterministic cryptographic procedure described in Section 3.2. If the same key k is used across multiple models $(\mathcal{M}_1, \ldots, \mathcal{M}_n)$, then the green-list G will be the same. As a result, the watermark proves that the output was generated under key k, but it cannot identify which model produced it. In this case, attribution is tied to the key (i.e., the watermarking authority), not to the underlying model.

To enable model-specific attribution, each model $\mathcal{M}_i$ must be assigned a distinct secret key k_i, yielding

$$G_i = \text{GenGreen}(k_i).$$

Since $k_i \neq k_j$ for $i \neq j$, the corresponding green-lists G_i and G_j are statistically independent. During detection, a given code sample $\hat{y}$ can be evaluated against multiple candidate keys $\{k_1, k_2, \ldots, k_m\}$, producing detection scores $\{S_1, S_2, \ldots, S_m\}$ (see Section 3.4). The attributed model index is then defined as

$$\hat{i} = \arg\max_i S_i,$$

provided that S_i exceeds a predefined significance threshold. This construction follows the same principle as digital signatures: if two systems use the same key, they are indistinguishable but if they use different keys, their outputs become uniquely identifiable. Thus, key separation enables model-specific attribution, while shared keys provide system-level attribution.

5.2 Robustness Under Maintenance

Watermark robustness under transformations is an important consideration, as prior works (Kim et al., 2025) has shown that watermarking schemes can degrade under paraphrasing or other semantic-preserving edits. To evaluate this aspect, we simulate two practical maintenance scenarios on the HumanEval dataset: (i) comment removal, and (ii) automated paraphrasing. We assume a black-box adversary who may have watermarked outputs and apply semantic-preserving transformations but does not have access to the secret key. To imitate the attack scenario, we utilized GPT-Codex-5.3 (OpenAI, 2026) with the specific instruction: *"Paraphrase the scripts and rename the variables where necessary keeping them relevant and natural."*

Table 3: Robustness of watermark detection under common maintenance attacks.

Attack Type	AUROC	Degradation	Degradation %
Baseline	0.9834	–	–
Without Comments	0.9456	-0.0378	-3.84%
Paraphrased	0.7139	-0.2695	-27.41%

Table 3 summarizes the detection performance under these transformations. On the original watermarked code, the detector achieves AUROC = 0.9834. After randomly stripping comments, the resulting AUROC decreases marginally to 0.9456 (-3.84%). This limited degradation indicates that the watermark signal is not concentrated in comments but is rather distributed across identifiers. However, automated paraphrasing results in a more substantial degradation, with AUROC dropping to 0.7139 (-27.41%). This behavior suggests that aggressive structural modifications, such as systematic variable renaming, can disrupt the statistical bias embedded in identifier initials and reduce detection strength.

These findings highlight an inherent trade-off. As the watermark is embedded through lexical biases rather than deep structural transformations, it remains resilient to routine maintenance operations such as partial comment stripping, whitespace adjustments, and limited variable renaming. However, the method is inherently vulnerable to large-scale refactoring attacks, such as systematically replacing all identifiers and comments. However, executing such a comprehensive attack incurs substantial manual effort and computational overhead, and risking the introduction of new functional errors.

Overall, PromptMark is robust under realistic, non-adversarial maintenance but remains vulnerable to targeted structural rewriting. In this research, our primary goal is to demonstrate black-box, prompt-

guided watermark embedding without logit manipulation. Enhancing robustness through multi-channel or redundancy-based strategies is left for future work.

5.3 Iteration Distribution and Cost Tradeoff

Figure 4 shows the iteration distribution for our generation procedure. Most tasks converge within the first two iterations, with only a small percentage requiring further refinement. This pattern aligns with prior work on iterative code refinement, such as Agent-Coder (Huang et al., 2023) , which also evaluates effectiveness across 1-5 iterations and finds incremental improvements with additional passes.

Figure 5 illustrates the total token usage aggregating input and output tokens per generation. It typically requires 800–900 tokens, which under standard pricing models of Claude Sonnet 4 translates into a negligible cost of approximately 1.4–1.6 cents per iteration. As cost scales with code length and complexity, and most tasks converge within a few iterations, the total expense remains minimal. Ultimately, this represents a pragmatic trade-off that the marginal computational cost is justified by the significant value of securing authorship attribution and verifiable code provenance.

6 THREATS TO VALIDITY

The validity of our results depends on how PromptMark uses LLM instruction-following behavior. This can cause variation in watermark strength as LLMs are inherently non-deterministic, producing inconsistent and different outputs on the same prompt. To reduce this effect, we run each experiment twice using two benchmark datasets and two different LLMs. We also use a structured system prompt with iterative feedback that further stabilizes both correctness and watermark strength across generations. On the other hand, to enable a more controlled analysis of watermarking behavior, we focus exclusively on the Python language following prior work (Zhang et al., 2025), while leaving generalizability to other programming languages as future work.

7 CONCLUSION AND FUTURE WORK

This study introduces a systematic approach that reframes black-box source code watermarking as a prompt-guided task. By shifting from intrusive logit manipulation to a statistically grounded, frequency-based stratification strategy, our approach cryptographically derives a key-dependent "green-list" analyzing human-authored code. This ensures the embedded signal is reproducible, secure, and linguistically natural. Coupled with an iterative-feedback loop, the LLM dynamically balances watermark fidelity with rigid syntactic constraints. Experimental results demonstrated that prompting can serve as an effective approach to embed verifiable watermarks, preserving code functionality and structure. While this work establishes a foundation for zero-bit prompt-guided watermarking, it opens complex avenues for future research. A major challenge is developing *encodable, multi-bit watermarking* exclusively for black-box setting. Furthermore, to tackle semantic-preserving adversarial attacks, future research should explore *multi-channel embedding* in order to achieve white-box-level robustness in black-box environments.

ACKNOWLEDGMENTS

During preparation, the author(s) used ChatGPT (OpenAI, 2026) for language refinement and restructuring. The author(s) reviewed and validated the content and take full responsibility for its accuracy, originality, and integrity.

REFERENCES

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. (2021). Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Bistarelli, S., Fiore, M., Mercanti, I., and Mongiello, M. (2025). Usage of large language model for code generation tasks: A review. *SN Computer Science*, 6(6).

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021). Evaluating large language models trained on code.

Chen, S., Brumby, D., and Cox, A. (2025). Envisioning the future of peer review: Investigating llm-assisted reviewing using chatgpt as a case study. In *Proceedings of the 4th Annual Symposium on Human-Computer Interaction for Work*, pages 1–18.

Collberg, C. and Thomborson, C. (2002). Watermarking, tamper-proofing, and obfuscation - tools for software protection. *IEEE Transactions on Software Engineering*, 28(8):735–746.

Dasgupta, A., Tanvir, A., and Zhong, X. (2024). Water-

marking language models through language models. *arXiv preprint arXiv:2411.05091*.

Dong, G., Lu, K., Li, C., Xia, T., Yu, B., Zhou, C., and Zhou, J. (2025). Self-play with execution feedback: Improving instruction-following capabilities of large language models. In *The Thirteenth International Conference on Learning Representations*.

Gu, J. (2025). Why is code provenance non-negotiable in the age of ai? Accessed: 2026-02-20.

Guan, B., Wan, Y., Bi, Z., Wang, Z., Zhang, H., Zhou, P., and Sun, L. (2024). CodeIP: A grammar-guided multi-bit watermark for large language models of code. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N., editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 9243–9258, Miami, Florida, USA. Association for Computational Linguistics.

Gurioli, A., Gabbrielli, M., and Zacchiroli, S. (2025). Is this you, llm? recognizing ai-written programs with multilingual code stylometry. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 394–405.

Hayes, J. and Danezis, G. (2017). Generating steganographic images via adversarial training. *Advances in neural information processing systems*, 30.

Huang, D., Zhang, J. M., Luck, M., Bu, Q., Qing, Y., and Cui, H. (2023). Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010*.

Kim, J., Park, S., and Han, Y.-S. (2025). Marking code without breaking it: Code watermarking for detecting llm-generated code. *arXiv preprint arXiv:2502.18851*.

Kirchenbauer, J., Geiping, J., Wen, Y., Katz, J., Miers, I., and Goldstein, T. (2023). A watermark for large language models. In *International Conference on Machine Learning*, pages 17061–17084. PMLR.

Knuth, D. E. (1997). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, Reading, MA, 3rd edition.

Lee, T., Hong, S., Ahn, J., Hong, I., Lee, H., Yun, S., Shin, J., and Kim, G. (2024). Who wrote this code? watermarking for code generation. In Ku, L.-W., Martins, A., and Srikumar, V., editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4890–4911, Bangkok, Thailand. Association for Computational Linguistics.

Li, B., Zhang, M., Zhang, P., Sun, J., Wang, X., and Fu, Z. (2024). Acw: Enhancing traceability of ai-generated codes based on watermarking. *arXiv preprint arXiv:2402.07518*.

Li, H., Zhang, J., Sun, X., and Luo, X. (2025). Codeguard: A generalized and stealthy backdoor watermarking for generative code models. *arXiv preprint arXiv:2506.20926*.

Li, W. (2011). Application of volcano plots in analyses of mrna differential expressions with microarrays. *arXiv preprint arXiv:1103.3434*.

Liu, H., Guo, M., Jiang, Z., Wang, L., and Gong, N. (2024). Audiomarkbench: Benchmarking robustness of audio watermarking. *Advances in Neural Information Processing Systems*, 37:52241–52265.

Liu, Y., Zhao, X., Kruegel, C., Song, D., and Bu, Y. (2025). In-context watermarks for large language models. In *ICML 2025 Workshop on Reliable and Responsible Foundation Models*.

Mahmud, T., Duan, B., Pasareanu, C., and Yang, G. (2025). Enhancing llm code generation with ensembles: A similarity-based selection approach. *arXiv preprint arXiv:2503.15838*.

Ning, K., Chen, J., Zhong, Q., Zhang, T., Wang, Y., Li, W., Zhang, J., Yu, J., Feng, Y., Zhang, W., et al. (2024). Mcgmark: An encodable and robust online watermark for tracing llm-generated malicious code. *arXiv preprint arXiv:2408.01354*.

OpenAI (2026). Introducing GPT-5.3-Codex. Accessed: 2026-02-23.

Paek, T. and Mohan, C. (2025). Detection of llm-generated java code using discretized nested bigrams. *arXiv preprint*.

Park, S., Jin, H. C. J.-w., et al. (2025). Detecting code paraphrased by large language models using coding style features. *Engineering Applications of Artificial Intelligence*, 162:112454.

Piantadosi, S. T. (2014). Zipf's word frequency law in natural language: A critical review and future directions. *Psychonomic bulletin & review*, 21(5):1112–1130.

Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., and Ma, S. (2020). Codebleu: a method for automatic evaluation of code synthesis.

Xiang, L., Li, N., Liu, Y., and Hu, J. (2025). Ai-generated text detection: A comprehensive review of active and passive approaches. *Computers, Materials Continua*, pages 1–10.

Xu, R., Chen, J., Yin, Z., Kong, C., and Zhang, X. (2025). Large language models are effective code watermarkers. *arXiv preprint arXiv:2510.11251*.

Xu, X. et al. (2024). Distinguishing llm-generated from human-written code by contrastive learning. *arXiv preprint*.

Yang, B., Li, W., Xiang, L., and Li, B. (2024). Srcmarker: Dual-channel source code watermarking via scalable code transformations. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4088–4106.

Yin, X. et al. (2025). Detecting llm-generated code with subtle modification by adversarial training. *arXiv preprint*.

Zhang, R., Javidnia, N., Sheybani, N., and Koushanfar, F. (2025). Robust and secure code watermarking for large language models via ml/crypto codesign. *arXiv preprint arXiv:2502.02068*.

Zhao, X., Gunn, S., Christ, M., Fairoze, J., Fabrega, A., Carlini, N., Garg, S., Hong, S., Nasr, M., Tramer, F., et al. (2025). Sok: Watermarking for ai-generated content. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 2621–2639. IEEE.